أكاديمية طويق
Tuwaiq Academy



UNIT 2 · ASSIGNMENT 2

Matrix Algebra & Systems of Equations

TEAM ASSIGNMENT REPORT

Prepared by

Raghad Alasmari

Rawan Alsubaie

Norah Alsuwailem

Fahad Al-Nahi

Instructor: Khulud Alshammari

Based on the accompanying solved Google Colab Notebook

Executive Summary

This report documents six missions on matrix algebra and systems of equations, combining manual calculations with numerical verification using NumPy. A small housing dataset (four houses described by Area, Bedrooms and Age) is represented as a 4 × 3 matrix, transformed with element-wise and structural operations, and multiplied by a weight vector to produce prediction scores of 250, 340, 410 and 500. Adding a redundant Rooms feature leaves the rank at 3, revealing one dependent column. Matrix A is singular (det = 0) while matrix B is invertible (det = 1), and the linear system 2x + y = 7, x − y = 2 has the unique solution x = 3, y = 1.

Aim and Method

The assignment requires: (1) manual calculations, (2) verification of every numerical result using NumPy, and (3) an answer to the AI Insight question in each mission. In the notebook every manual answer is checked with an `assert` statement, so any mismatch would stop execution. Matplotlib is used to illustrate the results. Rows of the data matrix are houses and columns are features (Area, Bedrooms, Age):

$$X = \begin{bmatrix} 120 & 3 & 10 \\ 150 & 4 & 8 \\ 180 & 4 & 6 \\ 210 & 5 & 4 \end{bmatrix}$$

Mission 1 — Understanding the Matrix

Each house becomes one row and each feature one column, giving the matrix X above. It has **4 rows** and **3 columns**, so its shape is **4 × 3** (NumPy: `X.shape == (4, 3)`).

Each row represents one house (a sample or observation): row 1, [120, 3, 10], is house H1 with an area of 120, 3 bedrooms and an age of 10 years. **Each column** represents one feature measured for every house: column 1, $[120, 150, 180, 210]^T$, is the Area of all four houses. The matrix is created in NumPy with `np.array([[120,3,10],[150,4,8],[180,4,6],[210,5,4]])`.

AI Insight | In Machine Learning the data is stored as a feature matrix of shape (n_samples, n_features). Rows are samples that the model learns from or predicts for, and columns are input features. The target (for example, Price) is kept separately as the label vector y. The model learns one weight per column.

Mission 2 — Matrix Operations

Each operation is applied to the original matrix X so that its effect can be seen on its own:

$$\text{Age} + 2 = \begin{bmatrix} 10+2 \\ 8+2 \\ 6+2 \\ 4+2 \end{bmatrix} = \begin{bmatrix} 12 \\ 10 \\ 8 \\ 6 \end{bmatrix} \qquad \text{Age} - 1 = \begin{bmatrix} 10-1 \\ 8-1 \\ 6-1 \\ 4-1 \end{bmatrix} = \begin{bmatrix} 9 \\ 7 \\ 5 \\ 3 \end{bmatrix}$$

$$1.1 \times \text{Area} = 1.1 \times \begin{bmatrix} 120 \\ 150 \\ 180 \\ 210 \end{bmatrix} = \begin{bmatrix} 132 \\ 165 \\ 198 \\ 231 \end{bmatrix} \qquad X^{\mathrm{T}} = \begin{bmatrix} 120 & 150 & 180 & 210 \\ 3 & 4 & 4 & 5 \\ 10 & 8 & 6 & 4 \end{bmatrix}$$

Applied one after the other, +2 and −1 would give a net change of +1, i.e. Age = [11, 9, 7, 5]. Multiplying Area by 1.1 is a 10% increase. The transpose swaps rows and columns, so the 4 × 3 matrix becomes 3 × 4:

each row is now a feature and each column a house. All four results were verified with NumPy (`X[:, 2] + 2`, `X[:, 2] - 1`, `X[:, 0] * 1.1`, `X.T`).

AI Insight | **Values** are changed by addition, subtraction and scalar multiplication; the shape stays 4 × 3. These mirror preprocessing steps such as centering and feature scaling. **Structure** is changed by the transpose, which keeps every value but turns 4 × 3 into 3 × 4. **Transpose is useful** because it makes shapes compatible for multiplication and appears in core formulas such as the covariance matrix X^TX / (n − 1) and the Normal Equation $w = (X^TX)^{-1}X^Ty$.

Mission 3 — AI Prediction

The weights Area = 2, Bedrooms = 20 and Age = −5 form a 3 × 1 weight matrix w. The negative Age weight lowers the score of older houses. Since the inner dimensions match, (4 × **3**)(**3** × 1) → (4 × 1):

$$\hat{y} = Xw = \begin{bmatrix} 120 & 3 & 10 \\ 150 & 4 & 8 \\ 180 & 4 & 6 \\ 210 & 5 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ 20 \\ -5 \end{bmatrix} = \begin{bmatrix} 240 + 60 - 50 \\ 300 + 80 - 40 \\ 360 + 80 - 30 \\ 420 + 100 - 20 \end{bmatrix} = \begin{bmatrix} 250 \\ 340 \\ 410 \\ 500 \end{bmatrix}$$

Each score is the dot product of one row with w; for H1: 120(2) + 3(20) + 10(−5) = 250. H4 (largest, most bedrooms, newest) receives the highest score and H1 the lowest. NumPy `X @ w` returns the same vector $[250, 340, 410, 500]^T$.

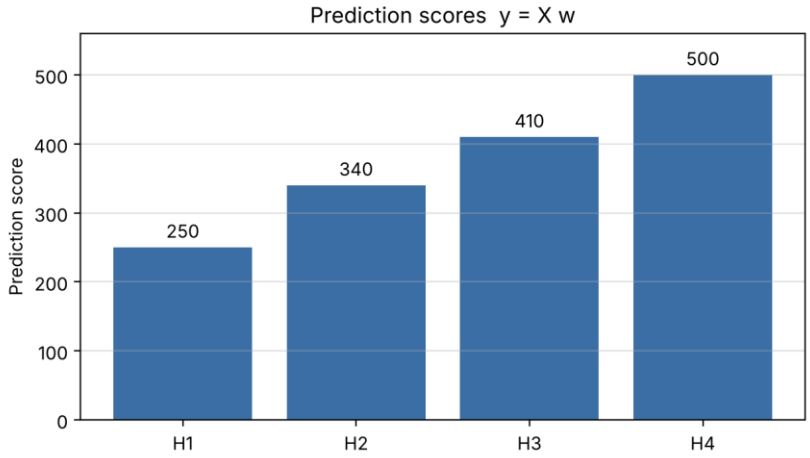


Figure 1. Prediction scores computed as y = Xw.

AI Insight | Matrix multiplication is the core computation of most models: linear and logistic regression predict all samples at once with Xw + b, every dense neural-network layer computes σ(XW + b), and transformer attention multiplies query, key and value matrices. Replacing loops with one matrix product (vectorization) is what makes training on GPUs fast.

Mission 4 — Detecting Redundant Features

Adding Rooms = 2 × Bedrooms as a fourth column gives a 4 × 4 matrix:

$$X_{\text{new}} = \begin{bmatrix} 120 & 3 & 10 & 6 \\ 150 & 4 & 8 & 8 \\ 180 & 4 & 6 & 8 \\ 210 & 5 & 4 & 10 \end{bmatrix} \qquad \text{col}_4 - 2 \cdot \text{col}_2 = 0$$

Manual rank. Because column 4 is exactly twice column 2, it adds no new direction, so the rank is at most 3 and det(X_{new}) = 0. To confirm that the remaining three columns are independent, a 3 × 3 sub-matrix (rows H1–H3, first three columns) must have a nonzero determinant:

$$\det\begin{bmatrix}120 & 3 & 10\\ 150 & 4 & 8\\ 180 & 4 & 6\end{bmatrix} = 120(-8) - 3(-540) + 10(-120) = -540 \neq 0$$

Therefore **rank(X_{new}) = 3** although the matrix has 4 columns. NumPy `np.linalg.matrix_rank(X_new)` returns 3, the minor evaluates to −540 and `np.linalg.det(X_new)` is 0.

Findings. One feature is redundant: Rooms can be predicted perfectly from Bedrooms, so it carries no new information and makes the matrix singular. The fix is to drop Rooms (or Bedrooms) and keep three features.

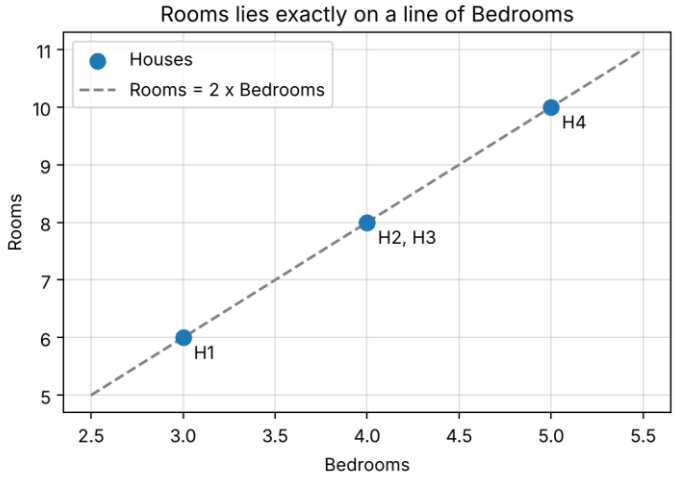


Figure 2. Every house lies exactly on the line Rooms = 2 × Bedrooms.

AI Insight | Redundant features cause **multicollinearity**: X^TX becomes singular, so linear regression has no unique solution and its weights become unstable and hard to interpret. They also waste computation and memory and raise the risk of overfitting without adding signal. Checking the rank, correlations or near-zero PCA eigenvalues detects them before training.

Mission 5 — Determinant, Identity Matrix & Inverse

For a 2 × 2 matrix, det = ad − bc.

$$\det A = \det\begin{bmatrix}2 & 1\\ 4 & 2\end{bmatrix} = 2(2) - 1(4) = \mathbf{0} \qquad \det B = \det\begin{bmatrix}2 & 1\\ 3 & 2\end{bmatrix} = 2(2) - 1(3) = \mathbf{1}$$

NumPy `np.linalg.det` returns 0.0 and 1.0. Multiplying by the identity leaves both matrices unchanged, which NumPy confirms with `np.allclose(A @ I, A)` and `np.allclose(B @ I, B)`:

$$AI = \begin{bmatrix}2 & 1\\ 4 & 2\end{bmatrix}\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix} = \begin{bmatrix}2 & 1\\ 4 & 2\end{bmatrix} = A \qquad BI = \begin{bmatrix}2 & 1\\ 3 & 2\end{bmatrix}\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix} = \begin{bmatrix}2 & 1\\ 3 & 2\end{bmatrix} = B$$

A matrix is invertible only if its determinant is nonzero, so **B is invertible** and **A is not**. The inverse of B, verified with `np.linalg.inv(B)` and B B^{-1} = I, is:

$$B^{-1} = \frac{1}{1}\begin{bmatrix}2 & -1\\ -3 & 2\end{bmatrix} = \begin{bmatrix}2 & -1\\ -3 & 2\end{bmatrix} \qquad BB^{-1} = \begin{bmatrix}4-3 & -2+2\\ 6-6 & -3+4\end{bmatrix} = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}$$

Why A has no inverse. The second row of A is twice the first ([4, 2] = 2[2, 1]), so A collapses the whole plane onto a single line. Many inputs map to the same output and the transformation cannot be undone; the inverse formula would require dividing by det(A) = 0. NumPy `np.linalg.inv(A)` raises *LinAlgError: Singular matrix*.

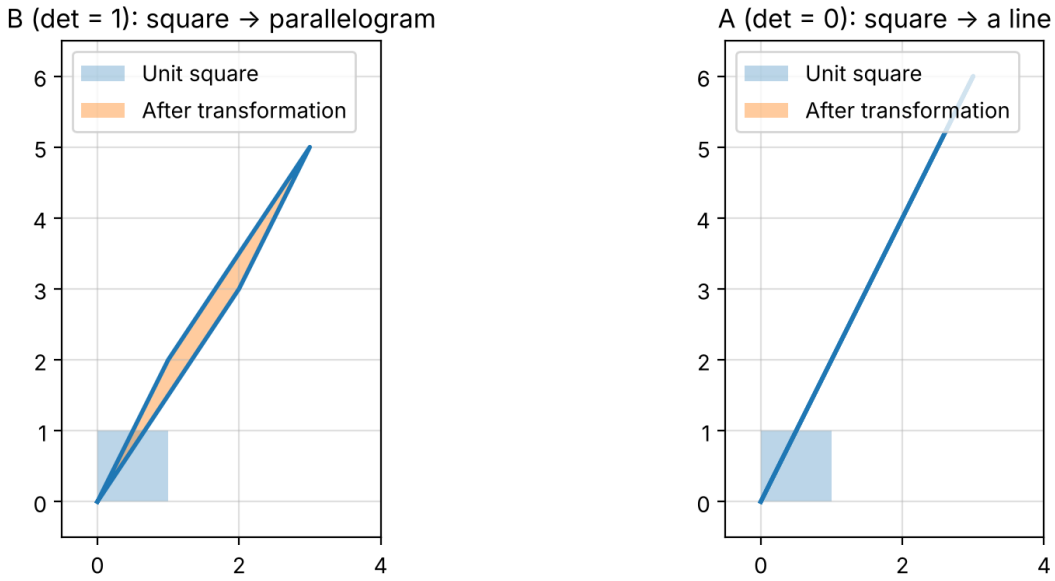


Figure 3. B maps the unit square to a parallelogram; A flattens it onto a line.

AI Insight | det = 0 means the matrix squashes space into a lower dimension: its rows are dependent and it is singular. **A singular matrix cannot be inverted** because information is lost and A^{-1} = adj(A) / det(A) divides by zero. **The inverse matters in ML** for the Normal Equation $w = (X^TX)^{-1}X^Ty$, for solving linear systems, and for the inverse covariance in Gaussian models. When X^TX is singular, Ridge regularization (adding λI) or the pseudo-inverse is used instead.

Mission 6 — Solving Linear Systems

The system 2x + y = 7 and x − y = 2 is written in matrix form *A***x** = **b**:

$$\begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 7 \\ 2 \end{bmatrix}$$

Elimination. Adding the equations removes y: 3x = 9, so **x = 3**; then 3 − y = 2 gives **y = 1**.

Inverse method. det(A) = 2(−1) − 1(1) = −3 ≠ 0, so:

$$A^{-1} = -\frac{1}{3}\begin{bmatrix} -1 & -1 \\ -1 & 2 \end{bmatrix} = \begin{bmatrix} 1/3 & 1/3 \\ 1/3 & -2/3 \end{bmatrix} \qquad \mathbf{x} = A^{-1}\mathbf{b} = \begin{bmatrix} 7/3 + 2/3 \\ 7/3 - 4/3 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

Check: 2(3) + 1 = 7 and 3 − 1 = 2. Both NumPy methods return the same answer:

Method	x	y
Manual — elimination	3	1
Manual — inverse matrix	3	1
NumPy `np.linalg.solve(A, b)`	3	1
NumPy `np.linalg.inv(A) @ b`	3	1

All four methods agree. Geometrically, (3, 1) is the single point where the two lines intersect. `np.linalg.solve` is preferred in practice because it is faster and more numerically stable than computing the inverse.

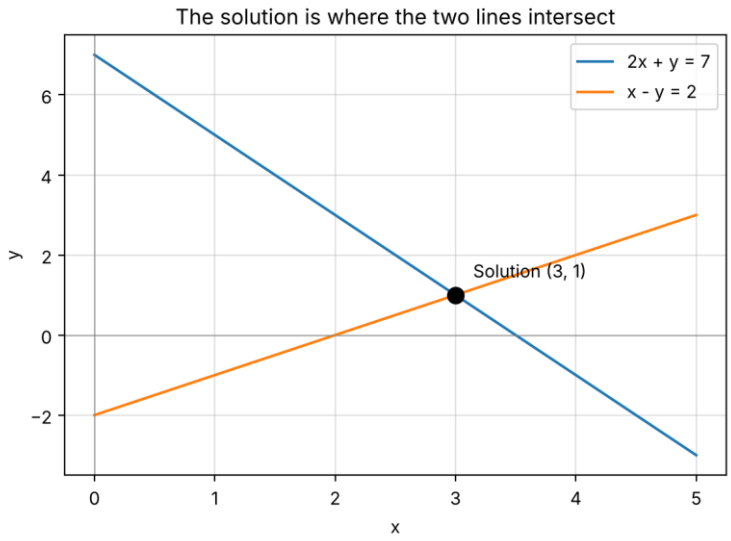


Figure 4. The solution is the intersection of the two lines.

AI Insight | Training linear regression is solving a linear system: the Normal Equation $(X^TX)w = X^Ty$ has the model weights as unknowns. Least squares, Gaussian Processes and Newton-type optimizers also solve Ax = b internally. Real datasets usually have more equations (samples) than unknowns (weights), so ML finds the best approximate solution that minimizes error. Whether a system has one, none or infinitely many solutions depends on the rank and determinant studied in Missions 4 and 5.

Conclusion

The assignment shows how a dataset becomes a matrix whose rows are samples and whose columns are features, how element-wise operations change values while the transpose changes structure, and how a single matrix product Xw produces predictions for every house at once ([250, 340, 410, 500]). Rank exposed the redundant Rooms feature (rank 3 with 4 columns), the determinant separated the singular matrix A from the invertible matrix B, and the inverse solved the linear system to give x = 3, y = 1. Manual and NumPy results agree in every mission.

Mission	Main result
1 — Understanding the Matrix	X is 4 × 3: rows = houses (samples), columns = features
2 — Matrix Operations	Age+2 = [12,10,8,6]; Age−1 = [9,7,5,3]; Area×1.1 = [132,165,198,231]; X^T is 3 × 4
3 — AI Prediction	Xw = $[250, 340, 410, 500]^T$
4 — Redundant Features	4 columns but rank 3: Rooms is redundant
5 — Determinant & Inverse	det A = 0 (singular); det B = 1; B^{-1} = [[2, −1], [−3, 2]]
6 — Linear Systems	x = 3, y = 1 (manual = NumPy)